

Министерство науки и высшего образования Российской Федерации

Муромский институт (филиал)

федерального государственного бюджетного образовательного учреждения

высшего

образования

«Владимирский государственный университет

имени Александра Григорьевича и Николая Григорьевича Столетовых»

Факультет: ИТР

Кафедра: ПИН

ЛАБОРАТОРНАЯ РАБОТА №1

По: Модульному тестированию

Тема: RomanNumeral Converter - арабские римские числа

Руководитель

Фамилия И.О.

(подпись)

(дата)

Студент группы ПИН-xxx

Фамилия И.О.

(подпись)

(дата)

Муром 2026

Лабораторная работа №1

Тема: RomanNumeral Converter - арабские римские числа

Цель работы: Освоить базовые навыки применения модульного тестирования с акцентом на выявление дефектов и оценку качества реализации программных компонентов.

1. Техническое задание и промпт

Промпт: Сгенерируй класс RomanNumeralConverter на Java со следующими методами: 1. Метод: toRoman, входные параметры: int number, возвращает: String. 2. Метод: toArabic, входные параметры: String roman, возвращает: int. Особые условия: метод toRoman должен выбрасывать IllegalArgumentException при значениях меньше 1 или больше 3999. Метод toArabic должен выбрасывать IllegalArgumentException при передаче пустой строки или некорректного формата римского числа. Класс должен реализовывать интерфейс IRomanNumeralConverter, находящийся в пакете lab.interfaces.

2. Исходный код

Реализации сгенерированы моделями Kimi K2, Model2 и Model3. Ниже представлены листинги исходного кода.

Реализация от gencode1:

```
package lab.implementations.gencode1;

import lab.interfaces.IRomanNumeralConverter;

/**
 * Converts between Arabic and Roman numerals.
 * Supported range: 1-3999.
 */
public class RomanNumeralConverter implements IRomanNumeralConverter {

    private static final int[] VALUES = {1000, 900, 500, 400, 100, 90,
        50, 40, 10, 9, 5, 4, 1};
```

```

private static final String[] SYMBOLS = {"M", "CM", "D", "CD", "C", "XC",
    "L", "XL", "X", "IX", "V", "IV", "I"};

@Override
public String toRoman(int number) {
    if (number < 1 || number > 3999) {
        throw new IllegalArgumentException(
            "Number must be in the range 1-3999 inclusive.");
    }

    StringBuilder sb = new StringBuilder();
    for (int i = 0; i < VALUES.length; i++) {
        while (number >= VALUES[i]) {
            sb.append(SYMBOLS[i]);
            number -= VALUES[i];
        }
    }
    return sb.toString();
}

@Override
public int toArabic(String roman) {
    if (roman == null || roman.isEmpty()) {
        throw new IllegalArgumentException("Roman numeral must not be null
or empty.");
    }

    String upper = roman.toUpperCase();
    int result = 0;
    int i = 0;

    while (i < upper.length()) {
        // Two-character symbols first
        if (i + 1 < upper.length()) {
            String two = upper.substring(i, i + 2);
            int idx = indexOf(two, SYMBOLS);
            if (idx != -1) {
                result += VALUES[idx];
                i += 2;
                continue;
            }
        }
        // Single-character symbols
        String one = upper.substring(i, i + 1);
        int idx = indexOf(one, SYMBOLS);
        if (idx == -1) {
            throw new IllegalArgumentException("Invalid Roman numeral: " +
roman);
        }
        result += VALUES[idx];
        i++;
    }
    // Validate canonical form by reconverting
    if (!toRoman(result).equals(upper)) {
        throw new IllegalArgumentException("Non-canonical Roman numeral: " +
roman);
    }
    return result;
}

```

```

        private static int indexOf(String symbol, String[] symbols) {
            for (int j = 0; j < symbols.length; j++) {
                if (symbols[j].equals(symbol)) {
                    return j;
                }
            }
            return -1;
        }
    }
}

```

Реализация от gencode2:

```

package lab.implementations.gencode2;

import lab.interfaces.IRomanNumeralConverter;

/**
 * Конвертер римских и арабских чисел.
 *
 * Методы:
 * - toRoman(int number): конвертирует арабское число (1..3999) в римскую
строку.
 * - toArabic(String roman): конвертирует корректную римскую строку в арабское
число.
 *
 * Бросает IllegalArgumentException при некорректных входных данных.
 */
public class RomanNumeralConverter implements IRomanNumeralConverter {

    private static final int[] VALUES = {
        1000, 900, 500, 400,
        100, 90, 50, 40,
        10, 9, 5, 4,
        1
    };

    private static final String[] SYMBOLS = {
        "M", "CM", "D", "CD",
        "C", "XC", "L", "XL",
        "X", "IX", "V", "IV",
        "I"
    };

    // Регулярное выражение для валидации корректного римского числа в диапазоне
1..3999
    private static final String ROMAN_REGEX =
        "^M{0,3}(CM|CD|D?C{0,3})(XC|XL|L?X{0,3})(IX|IV|V?I{0,3})$";

    @Override
    public String toRoman(int number) {
        if (number < 1 || number > 3999) {
            throw new IllegalArgumentException(
                "Аргумент number должен быть в диапазоне 1..3999
(включительно).");
        }

        StringBuilder sb = new StringBuilder();

```

```

        int remaining = number;

        for (int i = 0; i < VALUES.length; i++) {
            while (remaining >= VALUES[i]) {
                sb.append(SYMBOLS[i]);
                remaining -= VALUES[i];
            }
        }

        return sb.toString();
    }

    @Override
    public int toArabic(String roman) {
        if (roman == null) {
            throw new IllegalArgumentException("Аргумент roman не должен быть
null.");
        }

        String s = roman.trim();
        if (s.isEmpty()) {
            throw new IllegalArgumentException("Аргумент roman не должен быть
пустой строкой.");
        }

        if (!s.matches(ROMAN_REGEX)) {
            throw new IllegalArgumentException("Некорректный формат римского
числа: " + roman);
        }

        // Парсинг: суммируем, учитывая правило вычитания
        int total = 0;
        int i = 0;
        while (i < s.length()) {
            // Попробуем взять пару символов для возможного вычитания (например
            CM, IV и т.д.)
            if (i + 1 < s.length()) {
                String pair = s.substring(i, i + 2);
                int pairValue = valueOf(pair);
                if (pairValue > 0) {
                    total += pairValue;
                    i += 2;
                    continue;
                }
            }
            // Иначе берем одиночный символ
            String single = s.substring(i, i + 1);
            int singleValue = valueOf(single);
            // singleValue всегда > 0, так как строка заранее валидирована
            total += singleValue;
            i++;
        }

        return total;
    }

    // Возвращает значение римской последовательности (1 или 2 символа), либо 0
    если неизвестна
    private static int valueOf(String symbol) {

```

```

        switch (symbol) {
            case "M": return 1000;
            case "CM": return 900;
            case "D": return 500;
            case "CD": return 400;
            case "C": return 100;
            case "XC": return 90;
            case "L": return 50;
            case "XL": return 40;
            case "X": return 10;
            case "IX": return 9;
            case "V": return 5;
            case "IV": return 4;
            case "I": return 1;
            default: return 0;
        }
    }
}

```

Реализация от gencode3:

```

package lab.implementations.gencode3;

import lab.interfaces.IRomanNumeralConverter;
import java.util.HashMap;
import java.util.Map;
import java.util.regex.Pattern;

public class RomanNumeralConverter implements IRomanNumeralConverter {

    private static final Map<Character, Integer> ROMAN_TO_ARABIC = new
HashMap<>();
    private static final int[] ARABIC_VALUES = {1000, 900, 500, 400, 100, 90,
50, 40, 10, 9, 5, 4, 1};
    private static final String[] ROMAN_SYMBOLS = {"M", "CM", "D", "CD", "C",
"XC", "L", "XL", "X", "IX", "V", "IV", "I"};
    private static final Pattern VALID_ROMAN_PATTERN = Pattern.compile(
        "^M{0,3}(CM|CD|D?C{0,3})(XC|XL|L?X{0,3})(IX|IV|V?I{0,3})$"
    );

    static {
        ROMAN_TO_ARABIC.put('I', 1);
        ROMAN_TO_ARABIC.put('V', 5);
        ROMAN_TO_ARABIC.put('X', 10);
        ROMAN_TO_ARABIC.put('L', 50);
        ROMAN_TO_ARABIC.put('C', 100);
        ROMAN_TO_ARABIC.put('D', 500);
        ROMAN_TO_ARABIC.put('M', 1000);
    }

    @Override
    public String toRoman(int number) {
        if (number < 1 || number > 3999) {
            throw new IllegalArgumentException("Число должно быть в диапазоне от
1 до 3999");
        }
    }
}

```

```

        StringBuilder roman = new StringBuilder();
        int remaining = number;

        for (int i = 0; i < ARABIC_VALUES.length; i++) {
            while (remaining >= ARABIC_VALUES[i]) {
                roman.append(ROMAN_SYMBOLS[i]);
                remaining -= ARABIC_VALUES[i];
            }
        }

        return roman.toString();
    }

    @Override
    public int toArabic(String roman) {
        if (roman == null || roman.trim().isEmpty()) {
            throw new IllegalArgumentException("Римское число не может быть
пустым");
        }

        String trimmedRoman = roman.trim().toUpperCase();

        if (!isValidRoman(trimmedRoman)) {
            throw new IllegalArgumentException("Некорректный формат римского
числа: " + roman);
        }

        int result = 0;
        int previousValue = 0;

        for (int i = trimmedRoman.length() - 1; i >= 0; i--) {
            char currentChar = trimmedRoman.charAt(i);
            int currentValue = ROMAN_TO_ARABIC.get(currentChar);

            if (currentValue < previousValue) {
                result -= currentValue;
            } else {
                result += currentValue;
            }

            previousValue = currentValue;
        }

        return result;
    }

    private boolean isValidRoman(String roman) {
        return VALID_ROMAN_PATTERN.matcher(roman).matches();
    }
}

```

Анализ различий: Код от первой модели включает валидацию каноничной формы числа. Вторая модель не осуществляет приведение строки к верхнему регистру. Третья модель предоставляет базовую работоспособную логику.

3. Тесты

Ниже представлен исходный код модульных тестов.

```
package lab.tests;

import lab.implementations.gencode3.RomanNumeralConverter;
import lab.interfaces.IRomanNumeralConverter;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

public class GenCode1Tests {

    private final IRomanNumeralConverter converter = new
RomanNumeralConverter();

    // Тесты "черного ящика" (на основе спецификации)

    @Test
    public void toRoman_ValidInput_ReturnsRomanString() {
        // Arrange: инициализация тестовых данных
        int number = 10;

        // Act: вызов тестируемого метода
        String result = converter.toRoman(number);

        // Assert: проверка ожидаемого результата
        assertEquals("X", result);
    }

    @Test
    public void toRoman_BoundaryValues_ReturnsCorrectStrings() {
        // Проверка граничных значений из диапазона 1-3999
        assertEquals("I", converter.toRoman(1));
        assertEquals("MMMCMXCIX", converter.toRoman(3999));
    }

    @Test
    public void toRoman_OutOfRange_ThrowsIllegalArgumentException() {
        // Act & Assert: вызов метода с некорректными данными и проверка
        // исключения
        assertThrows(IllegalArgumentException.class, () -> {
            converter.toRoman(0);
        });

        assertThrows(IllegalArgumentException.class, () -> {
            converter.toRoman(4000);
        });
    }

    @Test
    public void toArabic_ValidInput_ReturnsArabicNumber() {
        // Arrange
        String roman = "MCMXCIV"; // 1994

        // Act
        int result = converter.toArabic(roman);
    }
}
```

```

        // Assert
        assertEquals(1994, result);
    }

    @Test
    public void toArabic_NullOrEmptyString_ThrowsIllegalArgumentException() {
        // Act & Assert: проверка реакции модуля на пустые входные данные
        assertThrows(IllegalArgumentException.class, () -> {
            converter.toArabic("");
        });

        assertThrows(IllegalArgumentException.class, () -> {
            converter.toArabic(null);
        });
    }

    // Тесты "белого ящика" (на основе анализа реализации Kimi K2)

    @Test
    public void toArabic_LowerCaseString_ProcessedSuccessfully() {
        // Этот сценарий не описан явно в спецификации, но анализ кода
        показывает
        // использование метода toUpperCase(), что означает поддержку нижнего
        регистра.

        // Arrange
        String lowerCaseRoman = "xiv"; // 14

        // Act
        int result = converter.toArabic(lowerCaseRoman);

        // Assert
        assertEquals(14, result);
    }

    @Test
    public void toArabic_NonCanonicalFormat_ThrowsIllegalArgumentException() {
        // Анализ кода: в методе toArabic есть проверка обратным преобразованием
        // (!toRoman(result).equals(upper)). Проверяем, что неканоничная запись
        "IIII"
        // вызовет ошибку, хотя алгоритмически это могло бы быть вычислено как
        4.

        // Arrange
        String nonCanonicalRoman = "IIII";

        // Act & Assert
        Exception exception = assertThrows(IllegalArgumentException.class, () ->
        {
            converter.toArabic(nonCanonicalRoman);
        });

        // Дополнительная проверка содержимого сообщения об ошибке
        assertTrue(exception.getMessage().contains("Non-canonical Roman
        numeral"));
    }

    @Test

```

```

    public void toArabic_InvalidCharacters_ThrowsIllegalArgumentException() {
        // Анализ кода: если символ не найден в массиве SYMBOLS, генерируется
        // исключение.

        // Arrange
        String invalidRoman = "XlV";

        // Act & Assert
        Exception exception = assertThrows(IllegalArgumentException.class, () ->
    {
        converter.toArabic(invalidRoman);
    });

        assertTrue(exception.getMessage().contains("Invalid Roman numeral"));
    }
}

```

Анализ падения тестов: Тесты GenCode2 и GenCode3 упали на проверке текста исключений ("Invalid Roman numeral" vs "Некорректный формат..."), так как модели сгенерировали сообщения на разных языках, а проверка assertThrows ожидала строгое совпадение строк. Также GenCode2 не прошел тест на обработку нижнего регистра (toArabic_LowerCaseString_ProcessedSuccessfully).

4. Выводы

Оценка качества: GenCode1 (5/5), GenCode2 (3/5), GenCode3 (4/5).

Тесты "белого ящика" выявили скрытый дефект во второй реализации (отсутствие нормализации регистра). Обнаружена проблема излишней жесткости самих тестов при проверке локализованных исключений.

Улучшение промпта: В промпт необходимо добавить требование поддержки любого регистра и строгого использования английского языка для генерации исключений.